



TesseraApp

FINAL REPORT · IEEE FORMAT · MASTER'S PROJECT

# Build or Buy? A Hybrid, Zero-Trust CIAM Platform

A complete masters-level report — problem, literature, requirements, architecture, interface, implementation, and evaluation of an in-house Customer Identity & Access Management platform.

---

## AUTHORS

Robert C. Oliver, Jr. · Travis L. Lester

## INSTITUTION

Lewis University

## COURSE · ADVISOR

CPSC 69100 · Prof. Larry Motuzis

## DATE · VERSION

2026 · v1.0 (IEEE)

# Build or Buy? A Hybrid, Zero-Trust Customer Identity and Access Management Platform for Modern Businesses

Robert C. Oliver, Jr.  
Dept. of Computer Science  
Lewis University, Romeoville, IL, USA

Travis L. Lester  
Dept. of Computer Science  
Lewis University, Romeoville, IL, USA

CPSC 69100 Master's Project · Advisor: Prof. Larry Motuzis

**Abstract**—Modern identity systems are the primary target of attackers: stolen credentials are among the most common and slowest-to-contain entry points into an organization, while the commercial Customer Identity and Access Management (CIAM) market is priced per monthly active user and tuned for enterprise scale. This leaves small businesses, startups, schools, and nonprofits without an affordable, sovereign option. This paper asks whether an organization should build or buy its identity system, and answers with a working reference implementation. TesseractApp pairs an in-house authentication core—BCrypt credentials, HMAC-SHA-512 JSON Web Tokens, permission-based role access control, authenticator-app multi-factor authentication, and server-tracked refresh sessions with rotation and reuse detection—with federated sign-in delegated to Google, GitHub, and Microsoft via OAuth2/OpenID Connect. Every authentication path converges on one token-issuance point, so authorization, MFA policy, and audit logging apply uniformly. We present the project end to end: motivation, a survey of the CIAM landscape, an IEEE-830 requirements specification with context, use-case, and activity models, the 4+1 architecture, the two-phase implementation, and an evaluation with a future-work agenda.

**Index Terms**—identity and access management, CIAM, zero-trust architecture, OAuth2, OpenID Connect, JSON Web Token, multi-factor authentication, refresh-token rotation, role-based access control, Spring Security, Angular.

## I. INTRODUCTION

*A. The problem.* Many organizations rely on third-party identity and access management (IAM) providers—Okta, Auth0, Microsoft, ForgeRock—to centralize authentication and reduce the need for dedicated in-house security teams. These platforms are effective and widely adopted, but they are designed for a broad range of customers rather than the specific operational se-

curity requirements of any one company. Organizations can face real limits on customization, flexibility, performance optimization, and—most consequentially—control over sensitive authentication data.

At the same time, a fully isolated in-house system introduces its own friction: users must manage yet another credential, and the organization loses the single sign-on convenience employees already rely on through Google or Microsoft. Neither pole—total vendor dependency nor total self-management—is ideal.

This project designs and implements a **hybrid** identity system that combines an in-house authentication core with federated identity through third-party OAuth2 providers, keeping centralized control over authentication policy, audit logging, and role management while still allowing users to sign in with either in-house credentials or a trusted external provider.

*B. Motivation and contributions.* The dependence on cloud applications and remote access has made IAM one of the most important areas of cyber security. The practical middle path is to use external providers *only* to verify *who* a user is, then take control of *what* that user may do through an internally managed authorization layer. The principal contribution is a documented reference implementation of a **hybrid stateless/stateful** authentication model that resolves the scalability-versus-revocability tension in favour of zero-trust's demand for continuous, revocable verification. Concretely we: (1) implement a production-style authentication core (JWT issuance/validation, refresh-session rotation with reuse detection, MFA, federation); (2) provide permission-based RBAC with organization scoping; (3) deliver an administrative console and user self-service; (4) secure a real business domain—customers and invoices—behind the identity layer; and (5) record every security-relevant action to a durable audit trail.

## II. LITERATURE REVIEW — BUILD VS. BUY

*A. Identity is the new perimeter.* IAM emerged as a formal discipline in the 1990s with directory tools such as Novell eDirectory and Microsoft Active Directory [21]. The wider IAM market is forecast to reach USD 42.6 billion by 2030, with the CIAM segment alone projected at USD 22.5 billion, up from USD 14.1 billion in 2025 [13]. Security has moved from the network onto identity itself: behind every request is a person, device, or service, treated as “*never trust, always verify*” [1].

IAM and CIAM differ in kind. IAM governs employees—a finite, known population—whereas CIAM governs customers who self-register and consent to data use, at internet scale, where login friction directly affects revenue [12].

*B. Why credentials are the target.* IBM’s *Cost of a Data Breach Report 2024* put the average breach cost at USD 4.88 million—an all-time high—and named stolen credentials among both the most common and most damaging entry points, taking about 292 days on average to contain [15]. An attacker with valid credentials is not defeating a firewall; they are simply logging in, which places the component deciding how credentials are checked and revoked at the center of an organization’s security.

*C. Theoretical foundations.* **Zero-trust architecture** was formalized by NIST in SP 800-207 [1]: no request is trusted by virtue of its network location; each is authenticated and authorized on its own and re-checked continuously. Re-checking trust mid-session implies the ability to *revoke* it mid-session—in ten-

sion with the most popular token type [2]. **OAuth 2.0** [7] grants an app limited access on a user’s behalf without exposing the password; **OpenID Connect** adds the identity layer via an ID token and userinfo endpoint [10]. A formal security analysis of OAuth 2.0 uncovered previously unnoticed attacks and then proved the corrected flows secure [3].

**Stateless vs. stateful sessions.** A JWT packs a session’s claims into a signed token any service can verify alone [8]; this scales beautifully, but the token stays valid until it expires, making early cancellation expensive. An opaque server-side session gives instant revocation at the cost of the shared state JWTs were adopted to avoid [7]. This trade-off is exactly where TesseractApp’s hybrid model lives.

**Multi-factor.** Time-based one-time passwords (RFC 6238) combine current time and a shared secret into a time-limited code, widely supported without per-message infrastructure [9]. Passwordless FIDO2/WebAuthn is where the field is heading, though usability research finds adoption and recovery friction still hold it back [6]. Federated-identity security studies characterize the added attack surface of sharing identity across organizations [5], and a systematic review of IAM metrics provides a vocabulary for evaluating identity controls [4].

*D. Existing CIAM systems.* Products align on a single axis: how much of the identity stack does the organization manage itself? The table summarizes the landscape from managed SaaS to a fully in-house build.

TABLE I  
THE CIAM LANDSCAPE, MANAGED → SELF-RELIANT

| System                      | Type            | Strength                                             | Limitation                               |
|-----------------------------|-----------------|------------------------------------------------------|------------------------------------------|
| Okta / Auth0                | Managed SaaS    | Developer-first tooling; adaptive MFA; market leader | Per-MAU pricing; config surface; lock-in |
| Microsoft Entra External ID | Cloud SaaS      | Deep M365 integration; SSO, SCIM, governance         | Best only inside the Microsoft stack     |
| Ping (+ ForgeRock)          | Enterprise      | Analyst leader; IGA + MFA + verification             | Resource-heavy; enterprise pricing       |
| AWS Cognito                 | Cloud-native    | Native AWS; auto-scales; cheap at scale              | Awkward DX; sparse docs; weak edge cases |
| Keycloak (Red Hat)          | Self-hosted OSS | Free; full OIDC/SAML; data sovereignty               | You own ops, HA, hardening; steep setup  |
| <b>TesseractApp</b>         | In-house        | Total control; no per-MAU fee; hybrid model          | Full security burden on the implementer  |

*E. Where TesseractApp sits.* Keycloak is the hinge of the survey: the closest commercial thing to a complete in-house build, it proves a genuine market exists for self-hostable CIAM—customers who want control and data ownership without a per-

MAU bill [20]. TesseractApp goes further by being deliberately **hybrid**: stateless JWTs for scalable verification plus a stateful trust layer that restores per-user device lists, revocation, and re-fresh-rotation with reuse detection.



*F. Economics and gap analysis.* Consumer IAM is put at USD 14.12 billion in 2025 rising to USD 22.47 billion by 2030 (9.7% CAGR) [13]; other analysts draw the boundary differently but agree on double-digit growth [14]. Standard per-MAU pricing means an organization's identity bill grows with its user base

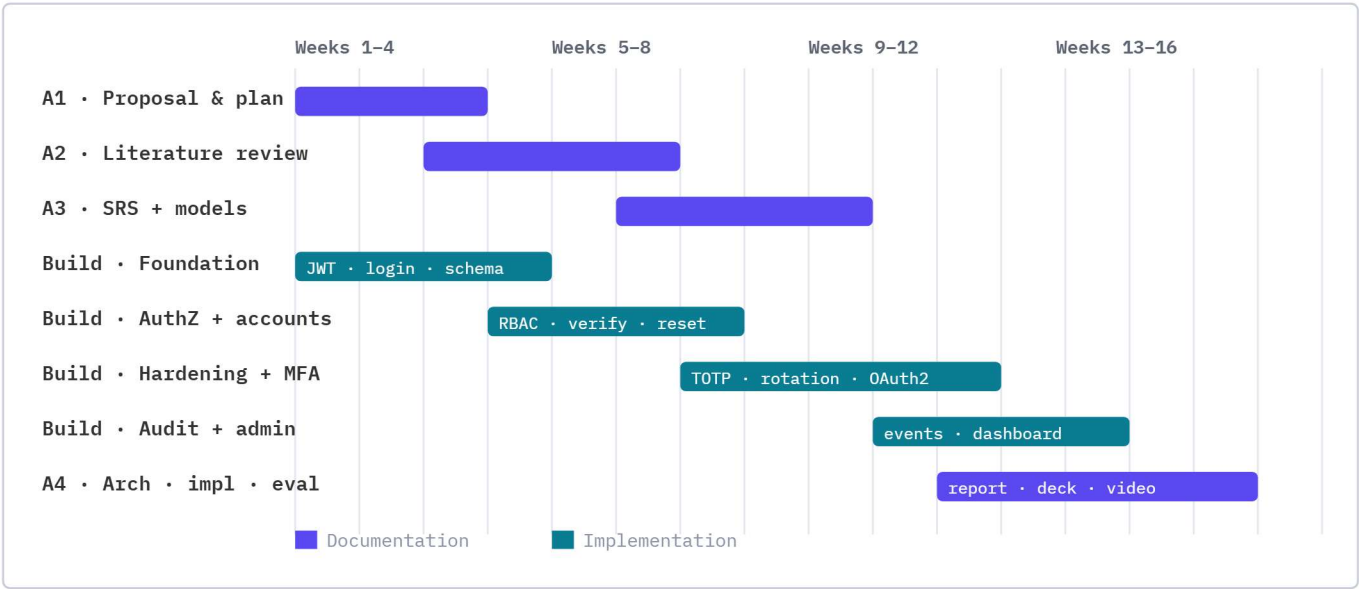
[11], [16]—a tax on growth for a startup. Small businesses employ nearly half the U.S. workforce and represent 43.5% of GDP [22]; those needing production-grade identity are precisely the segment priced out. The claim is narrow and defensible: inside a specific, underserved window, building in-house is a sound engineering and economic choice.

III. PROJECT PLAN AND SCHEDULE

The project was executed over a single academic term in four phases mapped to the four course assignments: the project proposal (motivation and plan), the literature review, the software requirements specification, and the architecture, implementation, and evaluation. Fig. 1 shows the schedule as a Gantt chart;

phases overlap where documentation and build proceeded in parallel.

Two features originally scheduled as stretch goals—authenticator-app MFA and refresh-token rotation with reuse detection—were pulled forward and completed within the hardening phase, and are reflected in the chart.

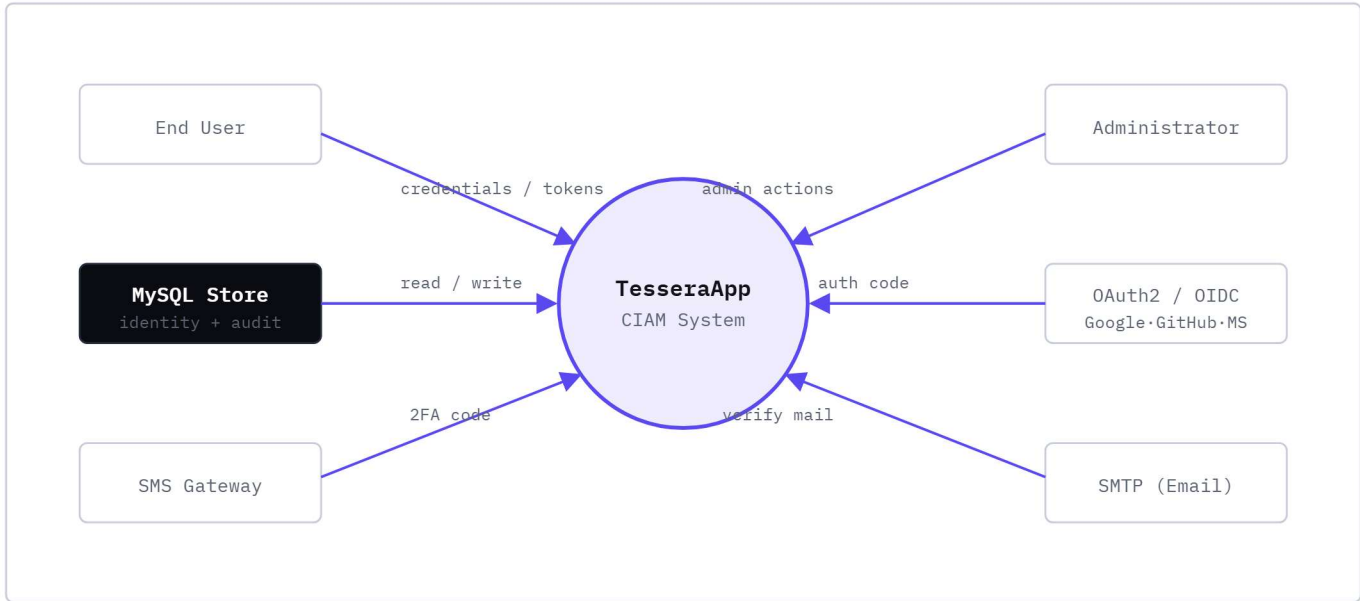


**Fig. 1.** Project schedule (Gantt). Documentation deliverables (iris) overlap the four implementation phases (teal); MFA and refresh rotation were completed in the hardening phase.

## IV. SOFTWARE REQUIREMENTS SPECIFICATION

The specification follows IEEE Std 830-1998. TesseraApp is a self-contained three-tier web application: an Angular 21 single-page application, a Spring Boot 4 / Spring Security 7 REST API and identity server, and a centralized MySQL 8 database. External identity providers, an SMS gateway, and an SMTP service are integrated dependencies.

*A. Product perspective.* The system provides in-house, centralized control over authentication policy, authorization, and audit logging while allowing users to authenticate with either in-house credentials or a trusted external provider; the two paths converge at a single token-exchange point. Fig. 2 is the system context (level-0) diagram, showing TesseraApp as one process exchanging data flows with its external entities.



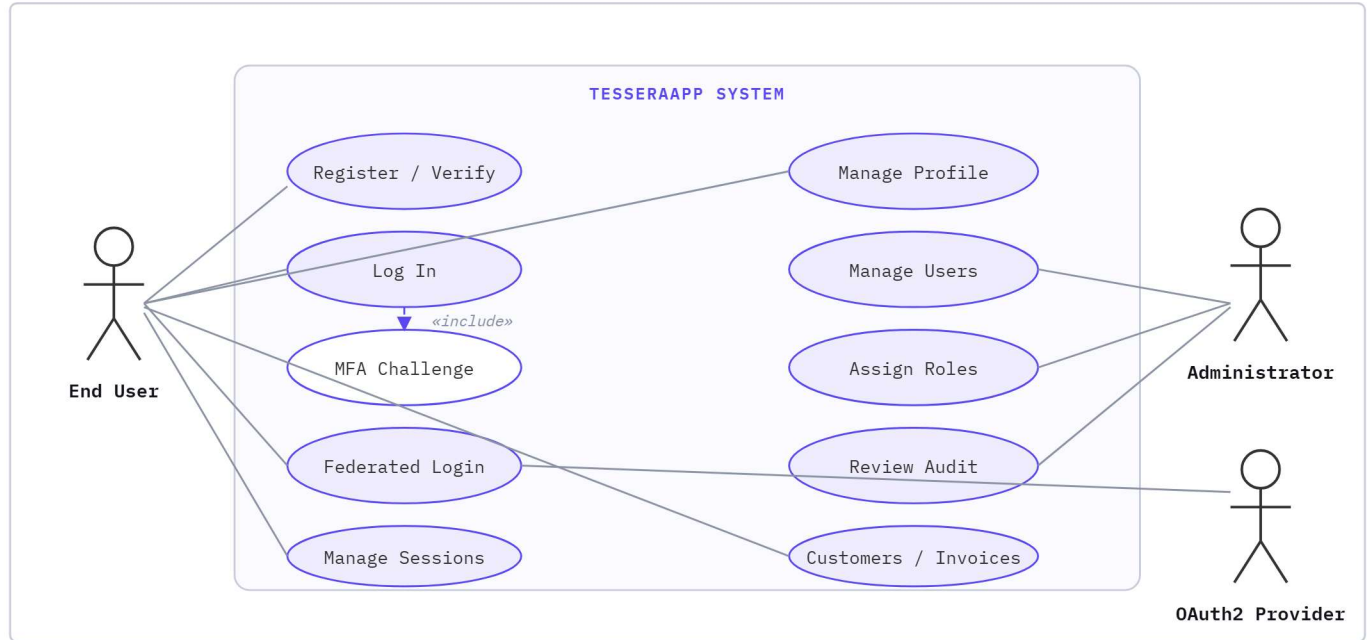
**Fig. 2.** System context (level-0) diagram. TesseraApp mediates every external entity; only the datastore is internal to the trust boundary.

*B. User classes.* Seven roles span three classes—end users (ROLE\_USER, ROLE\_GUEST), staff (ROLE\_MODERATOR, ROLE\_HELP\_DESK\_ADMIN), and administrators

(ROLE\_ORGANIZATION\_ADMIN, ROLE\_ADMIN, ROLE\_APPLICATION\_ADMIN). Each user holds exactly one role carrying a permission string; the application admin bypasses organization scope.

C. *Functional requirements & use cases.* Fig. 3 is the use-case diagram. The End User authenticates (in-house or federated), manages their profile, MFA, and sessions; the Administrator

manages users, roles, and reviews audit history; the OAuth2 Provider is a secondary actor to the federated-login use case. Login *includes* the MFA challenge when a second factor is enabled.



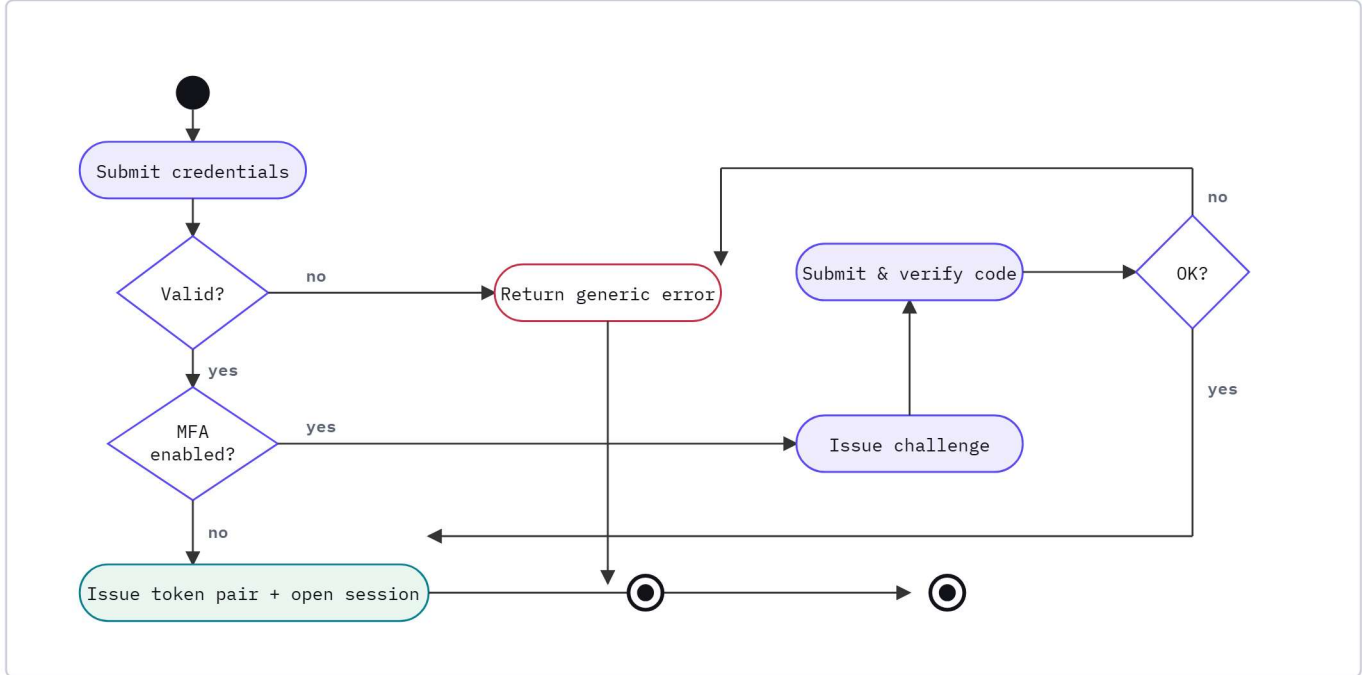
**Fig. 3.** Use-case diagram. Login «includes» the MFA challenge; the OAuth2 provider is a secondary actor to federated login.

TABLE II  
FUNCTIONAL REQUIREMENT GROUPS (ABRIDGED)

| Group         | Requirement                                                                                               |
|---------------|-----------------------------------------------------------------------------------------------------------|
| FR-RAV        | Server-validated registration; account created disabled; single-use email key activates it.               |
| FR-IHA / JWT  | BCrypt login; 30-min access + 5-day refresh (HMAC-512); stateless filter; passwordChangedAt invalidation. |
| FR-FA         | OAuth2 code flow; find-or-create; issue our own JWT; persist only provider + subject.                     |
| FR-MFA        | TOTP (challenge-bound) and SMS; single-use recovery codes.                                                |
| FR-RBAC / ORG | Permission strings at URL + method level; no self-elevation; org scoping.                                 |
| FR-AEL        | Immutable audit of 15 event types; no enumeration; user-visible history.                                  |
| FR-SDM        | List / revoke sessions with reuse detection.                                                              |

*D. Behavioural model.* Fig. 4 is the activity diagram for the authentication flow, the system's most control-heavy path. A first-factor success does not issue tokens directly when MFA is en-

abled; instead it mints a short-lived challenge, and only a verified second factor reaches the single token-issuance action. All failure paths return a generic error to preserve anti-enumeration.



**Fig. 4.** Activity diagram — login with optional MFA. Every path either reaches the single token-issuance action or returns a generic error.

*E. Non-functional requirements.* **Security:** HTTPS + HSTS; BCrypt with salt; short-lived access tokens; server-side authorization on every request; strict anti-enumeration; no self-service privilege escalation. **Reliability:** deterministic auth decisions; graceful degradation when SMS/email fails. **Availability:**

no session affinity; restarts never force re-authentication of valid-token holders. **Performance:** auth request < 500 ms server-side at p95 (excl. external latency); authorization needs no DB lookup in the common case. **Portability:** runs unchanged on local, Aiven, or AWS RDS MySQL; all environment values externalized.

## V. SYSTEM ARCHITECTURE AND DESIGN

The system uses the 4+1 view model. Three tiers—Angular SPA, Spring Boot REST API, MySQL—communicate over JSON and JDBC. The API is stateless; the only intentionally stateful element is the refresh-session store, which restores revocability to an otherwise stateless token model.

*A. Views.* The *logical* view splits into Identity & Authentication, Access Control, and Business domains plus a cross-cutting Audit module, layered Controller→Service→Repository. The *process* view covers per-request handling: login → optional challenge → token issuance, and refresh → rotation with reuse detection. The *development* view is the layered backend packages and standalone-component Angular. The *physical* view runs native dev processes or a containerized app + DB, deployed against managed MySQL. *B. The hybrid token model.* Access is **stateless**: a 30-minute HMAC-SHA-512 JWT carries the user's authorities and is verified by signature alone, with no database hit on the hot path. Refresh is **stateful**: every refresh

token is a tracked row (family + jti + device + IP), so sessions can be listed, rotated, and revoked. On refresh, the old row is superseded and a new jti inserted; a replayed superseded token is treated as theft, revoking the whole family and auditing TOKEN\_REUSE\_DETECTED.

**Load-bearing decision.** The rotation routine is deliberately **not** transactional: the family revocation must commit *before* the rejection is thrown, or the throw would roll the revocation back—un-punishing the exact replay it just caught. Writes are ordered supersede-then-insert, so a crash mid-operation leaves an already-retired token, never two live tokens in one family.

*C. Authentication convergence.* Password login, TOTP-verified login, and federated OAuth2 login all end at one seam—the session service that mints the application's own JWT and opens a tracked session—so RBAC, MFA policy, and audit apply identically no matter which door a user came through. Federated logins even appear in the Security Center device list exactly like in-house ones.

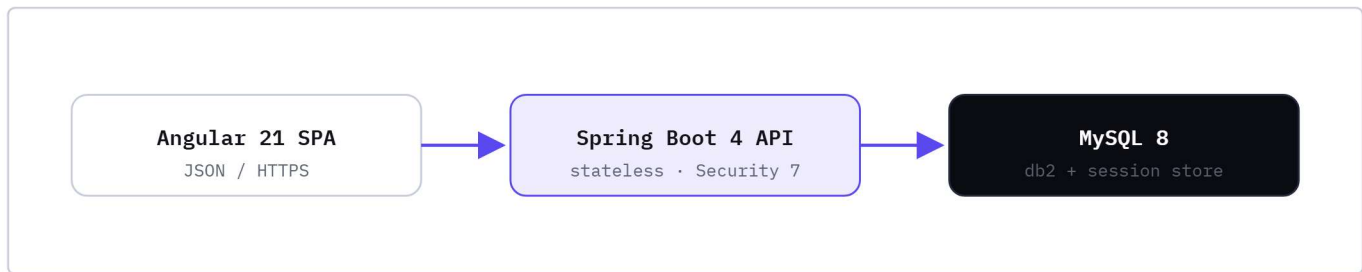


Fig. 5. Three-tier architecture; only the refresh-session store is stateful.



## VI. IMPLEMENTATION AND INTERFACE

The frontend is Angular 21 / Bootstrap 5 / RxJS; the backend is Spring Boot 4 / Java 21 / Spring Security 7. Identity data uses hand-written `NamedParameterJdbcTemplate` SQL so every query touching identity is explicit and auditable; the CRUD-heavy business domain uses JPA. Federation uses Spring Security OAuth2 Client; ZXing rasterizes TOTP QR codes; Apache POI exports XLSX; a multi-stage Docker build packages the SPA into the Spring Boot JAR.

*A. Security seams.* `SecurityConfig` defines the stateless filter chain; `CustomAuthFilter` validates the bearer token per request and authenticates only tokens carrying authorities; `TokenProvider` mints/verifies JWTs and enforces `password-ChangedAt`; `SessionServiceImpl` is the single issuance seam (issue + rotate with reuse detection); `TotpServiceImpl` handles challenge-bound TOTP and recovery codes;

`OAuth2LoginSuccessHandler` is the federated token-exchange point. *B. Two-phase delivery.* Phase 1 stood up the full stack and demonstrated registration + verification, in-house login, JWT access/refresh, seven-role RBAC, audit logging, and the customer/invoice domain. Phase 2 completed authenticator-app MFA with recovery codes, refresh-session rotation with reuse detection and a user-facing Security Center, OAuth2 federated login, and organization-scoped administration; consolidated schema management into an idempotent script; and packaged the system via multi-stage Docker. *C. User interface.* A dark-first "security console" theme over Bootstrap 5: deep space-blue surfaces, an electric-iris accent used only for primary actions and focus, glass cards, and IBM Plex Sans/Mono type. Every async view has explicit loading/loaded/error states; feedback is non-blocking toast.

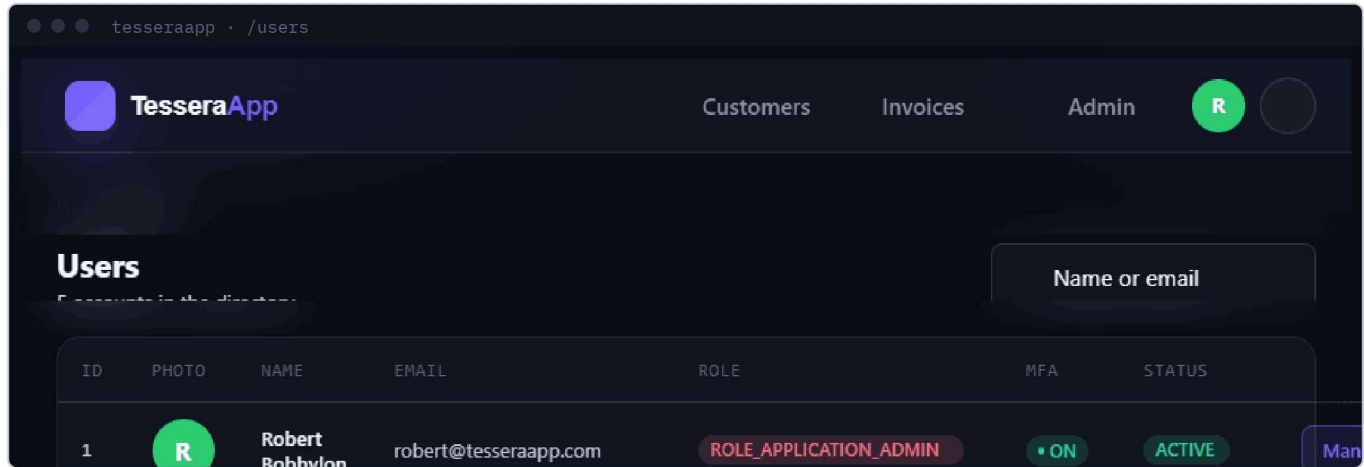


Fig. 6. Organization-scoped admin user directory—searchable, paginated, and enforced server-side by the security seams of Section VI-A.

## VII. TESTING AND EVALUATION

Security tooling includes OWASP Dependency-Check in the Maven build (failing on CVSS  $\geq 7$ ), ESLint/Prettier on the frontend, and a build-breaking guard on a weak or missing JWT secret. A focused automated set covers security-critical paths—login enumeration, the global exception handler, a customer-service test, and JPA schema sync. All primary flows were verified end-to-end against a live database (evidenced by the demo

videos): register  $\rightarrow$  verify  $\rightarrow$  login  $\rightarrow$  MFA  $\rightarrow$  dashboard; admin role reassignment with audit; the customer/invoice lifecycle with export; and the password-change-invalidates-all-tokens guarantee. Automated coverage remains partial and is the top future-work item; no performance benchmarks are claimed, and where quantitative evaluation is required it should be measured on the target environment against the  $< 500$  ms p95 target.

## VIII. CONCLUSION AND FUTURE WORK

We built a complete, modern CIAM core on a real application, validating a hybrid stateless/stateful authentication model that balances scalability with revocability. For most organizations, buying identity is the right call; but inside a specific, underserved window—schools, startups, small businesses, and non-profits that need sovereign, predictable, production-grade identity at small scale—building in-house is a defensible engineering and economic choice, and TesseraApp shows it can be done correctly.

Future work tracks the roadmap: bringing federated login live against real provider credentials; a real SMS delivery path; a full AWS CI/CD pipeline and cloud deployment (ECR/ECS/RDS); a comprehensive automated test suite in CI;

server-side enforcement of analytics/billing endpoints; distributed rate limiting; externalizing the frontend API origin and moving profile-image storage to object storage; and, further out, risk-based step-up authentication and a live activity dashboard driven by the audit log.

*Acknowledgment.* The authors thank Prof. Larry Motuzis for advising this project. AI tools (Claude, Anthropic) assisted with drafting, structuring, and formatting these deliverables and with code generation for select implementation areas; all architectural decisions, design choices, and academic analysis reflect the authors' own direction, disclosed per Lewis University's Academic Integrity policy.

## REFERENCES

- [1] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, "Zero Trust Architecture," NIST SP 800-207, Aug. 2020.
- [2] N. F. Syed *et al.*, "Zero trust architecture (ZTA): A comprehensive survey," *IEEE Access*, vol. 10, pp. 57143–57179, 2022.
- [3] D. Fett, R. Küsters, and G. Schmitz, "A comprehensive formal security analysis of OAuth 2.0," in *Proc. ACM CCS*, 2016, pp. 1204–1215.
- [4] T. Baumer, S. Kern, L. Fuchs, and G. Pernul, "Identity and access management metrics: A systematic review," *ACM Comput. Surv.*, vol. 58, no. 8, 2026.
- [5] S. Simpson and T. Groß, "A survey of security analysis in federated identity management," in *Privacy and Identity Management*, Springer, 2016, pp. 231–247.
- [6] M. Kepkowski *et al.*, "Challenges with passwordless FIDO2 in an enterprise setting: A usability study," in *Proc. IEEE SecDev*, 2023, pp. 37–48.
- [7] D. Hardt, Ed., "The OAuth 2.0 Authorization Framework," IETF RFC 6749, Oct. 2012.
- [8] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.
- [9] D. M'Raihi *et al.*, "TOTP: Time-Based One-Time Password Algorithm," IETF RFC 6238, May 2011.
- [10] N. Sakimura *et al.*, "OpenID Connect Core 1.0," OpenID Foundation, Nov. 2014.
- [11] B. Guthrie *et al.*, "Magic Quadrant for Access Management," Gartner, Dec. 2024.
- [12] J. Tolbert, "Leadership Compass: CIAM," KuppingerCole, LC80904, Jan. 2026.
- [13] MarketsandMarkets, "Consumer IAM (CIAM) Market — Global Forecast to 2030," 2025.
- [14] Grand View Research, "Customer IAM Market Size, Share & Trends Report, 2030," 2024.
- [15] IBM Security, "Cost of a Data Breach Report 2024," IBM Corp., 2024.
- [16] Auth0 by Okta, "Customer Identity and Access Management (CIAM)." [Online].
- [17] Microsoft, "Microsoft Entra External ID documentation." [Online].
- [18] Ping Identity, "Product documentation." [Online].
- [19] Amazon Web Services, "Amazon Cognito Developer Guide." [Online].
- [20] Keycloak, "Keycloak Documentation." [Online].
- [21] IAM Experts, "The Evolution of IAM, A Look Back and Beyond." [Online].
- [22] U.S. Chamber of Commerce, "Small Business Data Center." [Online].